

PW EARNERS
DSA in C++

DPP — 5 (SOLUTIONS)

Topic: Functions and Pointers in C++

For Q1 to Q15: Predict the exact output of each C++ program block. Read the concepts carefully!

Q1.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 10;
5     {
6         int x = 20;
7         cout << x << " ";
8     }
9     cout << x;
10 }
```

Output:

20 10

Explanation: The inner block creates a new local scope. The inner x (20) shadows the outer x. It prints 20. Once the block ends, the inner x is destroyed, and the outer x (10) is printed.

.....

Q2.

```
1 #include<iostream>
2 using namespace std;
3 int val = 50;
4 int main(){
5     int val = 15;
6     cout << val << " " << ::val;
7 }
```

Output:

15 50

Explanation: val accesses the local variable (15), while ::val uses the scope resolution operator to explicitly access the global variable (50).

.....

Q3.

```
1 #include<iostream>
2 using namespace std;
3 void printDetails(int a, int b = 10, int c = 20) {
4     cout << a << "-" << b << "-" << c << endl;
```

```

5 }
6 int main(){
7     printDetails(5);
8     printDetails(5, 15);
9 }

```

Output:

5-10-20

5-15-20

Explanation: In the first call, `b` and `c` use default arguments. In the second call, `b` is overwritten with 15, while `c` retains its default argument of 20.

Q4.

```

1 #include<iostream>
2 using namespace std;
3 void modify(int x) {
4     x = x + 10;
5 }
6 int main(){
7     int a = 5;
8     modify(a);
9     cout << a;
10 }

```

Output:

5

Explanation: The function accepts `x` by value. A copy is modified inside the function, leaving the original variable `a` in `main` unchanged.

Q5.

```

1 #include<iostream>
2 using namespace std;
3 void modifyRef(int &x) {
4     x = x + 10;
5 }
6 int main(){
7     int a = 5;
8     modifyRef(a);
9     cout << a;
10 }

```

Output:

15

Explanation: The function accepts `x` by reference (`&`). Modifying `x` directly modifies the original variable `a` in `main`.

Q6.

```

1 #include<iostream>
2 using namespace std;

```

```

3 int main(){
4     int score = 40;
5     int* ptr = &score;
6     *ptr = 99;
7     cout << score;
8 }

```

Output:

99

Explanation: ptr holds the memory address of score. Dereferencing it (*ptr) and assigning 99 directly changes the value of score in memory.

.....

Q7.

```

1 #include<iostream>
2 using namespace std;
3 void display(int a) { cout << "Int: " << a << endl; }
4 void display(double a) { cout << "Double: " << a << endl; }
5 int main(){
6     display(7);
7     display(7.5);
8 }

```

Output:

Int: 7

Double: 7.5

Explanation: Function overloading allows multiple functions with the same name but different parameter types. The compiler correctly resolves 7 to the int version and 7.5 to the double version.

.....

Q8.

```

1 #include<iostream>
2 #include<cmath>
3 using namespace std;
4 int main(){
5     int x = 36, y = 15;
6     cout << max((int)sqrt(x), y);
7 }

```

Output:

15

Explanation: sqrt(36) evaluates to 6. The max(6, 15) function compares them and returns the larger value, which is 15.

.....

Q9.

```

1 #include<iostream>
2 using namespace std;
3 int square(int n) { return n * n; }
4 int addFive(int n) { return square(n) + 5; }

```

```

5 int main(){
6     cout << addFive(4);
7 }

```

Output:

21

Explanation: addFive(4) calls square(4). square(4) returns 16. Then 5 is added, resulting in 21.

.....

Q10.

```

1 #include<iostream>
2 using namespace std;
3 void update(int* p) {
4     *p = *p * 2;
5 }
6 int main(){
7     int x = 12;
8     update(&x);
9     cout << x;
10 }

```

Correct Option: B

Explanation: The address of x is passed to update(). The function dereferences it and multiplies the value by 2 ($12 \times 2 = 24$), updating x in main.

.....

Q11.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int a = 8;
5     int* p1 = &a;
6     int** p2 = &p1;
7     cout << **p2;
8 }

```

Correct Option: C

Explanation: p1 stores the address of a. p2 stores the address of p1. Dereferencing p2 twice (**p2) jumps through both pointers to fetch the original value of a (8).

.....

Q12.

```

1 #include<iostream>
2 using namespace std;
3 int num = 100;
4 void compute(int num) {
5     num = num + ::num;
6     cout << num << " ";
7 }
8 int main(){
9     compute(20);

```

```

10     cout << num;
11 }

```

Output:

120 100

Explanation: Inside `compute`, the local `num` (20) is added to the global `::num` (100), making the local `num` 120 (which is printed first). The global `num` in `main` remains unchanged at 100.

.....

Q13.

```

1  #include<iostream>
2  using namespace std;
3  int main(){
4      int x = 10, y = 20;
5      int *p = &x;
6      p = &y;
7      *p = 50;
8      cout << x << " " << y;
9  }

```

Output:

10 50

Explanation: Pointer `p` initially points to `x`, but is reassigned to point to `y`. Dereferencing it and assigning 50 changes `y` to 50. `x` remains 10.

.....

Q14.

```

1  #include<iostream>
2  using namespace std;
3  int globalVar = 10;
4  void mysteryFunction(int* ptr, int &ref) {
5      *ptr = ref + globalVar;
6      ref = *ptr - 5;
7      globalVar++;
8  }
9  int main(){
10     int a = 20, b = 30;
11     mysteryFunction(&a, b);
12     cout << a << " " << b << " " << globalVar;
13 }

```

Output:

40 35 11

Explanation: 1. `ptr` points to `a`. `ref` is a reference to `b`.

2. `*ptr = 30 + 10 = 40`. So, `a` becomes 40.

3. `ref = 40 - 5 = 35`. So, `b` becomes 35.

4. `globalVar` increments to 11.

.....

Q15.

```

1  #include<iostream>

```

```

2 using namespace std;
3 int& trickyUpdate(int &x, int y) {
4     x = x + y;
5     return x;
6 }
7 int main(){
8     int val = 5;
9     trickyUpdate(val, 3) = 25;
10    cout << val;
11 }

```

Output:

25

Explanation: The function returns a reference to `x` (which refers to `val`). The evaluation `trickyUpdate(val, 3)` turns `val` into 8 and returns it. The assignment `= 25` immediately overwrites `val` with 25.

.....

Q16. State **True** or **False** for each statement below regarding C++ Functions and Pointers.

- (i) Default arguments in a C++ function must always be provided from right to left. **True**
- (ii) The statement `int* p = &x;` stores the value of variable `x` directly inside `p`. **False**
- (iii) Function overloading allows you to have two functions with the exact same name and parameter list, as long as their return types are different. **False**
- (iv) Using the scope resolution operator `::x` allows you to access a global variable `x` even if a local variable `x` is shadowing it. **True**
- (v) A double pointer `int** p` is used to store the address of a regular integer variable directly. **False**
- (vi) The ampersand operator (`&`) is used to get the memory address of a variable. **True**
- (vii) The inbuilt math function `sqrt(25)` returns an integer data type by default in C++. **False**

For Q17 to Q30: Write clean, modular C++ code. Use proper functions wherever instructed. Handle input/output gracefully.

Q17.

```

1 #include <iostream>
2 #include <algorithm>
3 using namespace std;
4
5 int findMaximum(int a, int b, int c) {
6     return max(a, max(b, c));
7 }
8

```

```

9  int main() {
10     int a, b, c;
11     cin >> a >> b >> c;
12     cout << findMaximum(a, b, c) << endl;
13 }

```

Explanation: The inbuilt `max()` function takes two arguments. We nest it `max(a, max(b, c))` to easily compare three values and return the largest.

.....

Q18.

```

1  #include <iostream>
2  using namespace std;
3
4  void printEven(int start, int end) {
5      for(int i = start; i <= end; i++) {
6          if(i % 2 == 0) cout << i << " ";
7      }
8  }
9
10 int main() {
11     int start, end;
12     cin >> start >> end;
13     printEven(start, end);
14 }

```

Explanation: We iterate from `start` to `end`. The modulo operator `% 2 == 0` checks if the number is divisible by 2, and prints it if true.

.....

Q19.

```

1  #include <iostream>
2  using namespace std;
3
4  void swapNumbers(int* a, int* b) {
5      int temp = *a;
6      *a = *b;
7      *b = temp;
8  }
9
10 int main() {
11     int x, y;
12     cin >> x >> y;
13     swapNumbers(&x, &y);
14     cout << x << " " << y << endl;
15 }

```

Explanation: By passing memory addresses using pointers, we create a `temp` variable to hold the value temporarily, allowing us to safely swap the values in memory.

.....

Q20.

```
1 #include <iostream>
2 using namespace std;
3
4 double calculateArea(double radius) {
5     return 3.14 * radius * radius;
6 }
7
8 double calculateArea(double length, double width) {
9     return length * width;
10 }
11
12 int main() {
13     cout << calculateArea(3.0) << endl;
14     cout << calculateArea(4.0, 5.0) << endl;
15 }
```

Explanation: Both functions share the exact same name, but take different parameters. C++ automatically figures out whether to calculate the area of a circle or rectangle based on how many arguments are provided.

.....
Q21.

```
1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4
5 double calculateHypotenuse(double base, double perp) {
6     return sqrt((base * base) + (perp * perp));
7 }
8
9 int main() {
10     double b, p;
11     cin >> b >> p;
12     cout << calculateHypotenuse(b, p) << endl;
13 }
```

Explanation: Pythagoras theorem $h = \sqrt{b^2 + p^2}$ is mapped into code using the mathematical functions <cmath> library.

.....
Q22.

```
1 #include <iostream>
2 using namespace std;
3
4 int sumFirstAndLast(int n) {
5     int lastDigit = n % 10;
6     while(n >= 10) {
7         n /= 10;
8     }
9     int firstDigit = n;
10    return firstDigit + lastDigit;
11 }
```

```

12
13 int main() {
14     int num;
15     cin >> num;
16     cout << sumFirstAndLast(num) << endl;
17 }

```

Explanation: We extract the last digit using modulo 10. To find the first digit, we continuously divide the number by 10 until only one digit remains.

.....

Q23.

```

1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4
5 void updateValues(int *a, int *b) {
6     int originalA = *a;
7     int originalB = *b;
8     *a = originalA + originalB;
9     *b = abs(originalA - originalB);
10 }
11
12 int main() {
13     int a, b;
14     cin >> a >> b;
15     updateValues(&a, &b);
16     cout << "a = " << a << ", b = " << b << endl;
17 }

```

Explanation: Since we must mutate both variables, we save their initial dereferenced states in originalA and originalB. This prevents overwriting data mid-calculation. `abs()` handles the absolute difference.

.....

Q24.

```

1 #include <iostream>
2 using namespace std;
3
4 int factorial(int x) {
5     int fact = 1;
6     for(int i = 1; i <= x; i++) fact *= i;
7     return fact;
8 }
9
10 int nCr(int n, int r) {
11     return factorial(n) / (factorial(r) * factorial(n - r));
12 }
13
14 int main() {
15     int n;
16     cin >> n;

```

```

17     for(int i = 0; i < n; i++) {
18         for(int j = 0; j <= i; j++) {
19             cout << nCr(i, j) << " ";
20         }
21         cout << endl;
22     }
23 }

```

Explanation: Pascal's triangle elegantly maps to combinations. Element j in row i is mathematically equivalent to exactly $iCr(j)$. The helper functions compute this efficiently.

.....

Q25.

```

1  #include <iostream>
2  using namespace std;
3
4  int reverseNumber(int n) {
5      int rev = 0;
6      while(n > 0) {
7          rev = rev * 10 + (n % 10);
8          n /= 10;
9      }
10     return rev;
11 }
12
13 int main() {
14     int num;
15     cin >> num;
16     cout << reverseNumber(num) << endl;
17 }

```

Explanation: We build the reversed integer by extracting the last digit ($n \% 10$) and appending it to rev via multiplication by 10.

.....

Q26.

```

1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  bool isPrime(int n) {
6      if(n <= 1) return false;
7      for(int i = 2; i <= sqrt(n); i++) {
8          if(n % i == 0) return false;
9      }
10     return true;
11 }
12
13 int main() {
14     int n;
15     cin >> n;
16     for(int i = 1; i <= n; i++) {

```

```

17         if(isPrime(i)) cout << i << " ";
18     }
19 }

```

Explanation: isPrime checks divisibility up to $\sqrt{n}$ for efficiency. The loop in main filters the numbers from 1 to N through this boolean check.

.....

Q27.

```

1  #include <iostream>
2  using namespace std;
3
4  void changeTarget(int** ptr, int* newAddress) {
5      *ptr = newAddress;
6  }
7
8  int main() {
9      int a = 10, b = 20;
10     int *p = &a;
11     changeTarget(&p, &b);
12     cout << *p << endl;
13 }

```

Explanation: We pass the address of pointer p (making it an int** in the function). By dereferencing it once (*ptr), we can reassign p to point to the address of b.

.....

Q28.

```

1  #include <iostream>
2  using namespace std;
3
4  double balance = 0.0; // Global Variable
5
6  void deposit(double amount) {
7      balance += amount;
8  }
9
10 void withdraw(double amount) {
11     if(balance >= amount) balance -= amount;
12 }
13
14 int main() {
15     deposit(500);
16     withdraw(200);
17     cout << "Balance: " << balance << endl;
18 }

```

Explanation: balance is declared outside any function, making it globally accessible. Both deposit and withdraw can freely modify this shared state without passing parameters.

.....

Q29.

```
1 #include <iostream>
2 using namespace std;
3
4 void sortTwo(int &a, int &b) {
5     if (a > b) {
6         int temp = a;
7         a = b;
8         b = temp;
9     }
10 }
11
12 int main() {
13     int x, y;
14     cin >> x >> y;
15     sortTwo(x, y);
16     cout << x << " " << y << endl;
17 }
```

Explanation: By passing variables by reference (&), we can directly manipulate `main`'s memory. If the first variable is larger, we swap them to guarantee sorted order.

.....

Q30.

```
1 #include <iostream>
2 using namespace std;
3
4 int getFactorial(int x) {
5     int fact = 1;
6     for(int i = 1; i <= x; i++) fact *= i;
7     return fact;
8 }
9
10 int findPermutation(int n, int r) {
11     return getFactorial(n) / getFactorial(n - r);
12 }
13
14 int main() {
15     int n, r;
16     cin >> n >> r;
17     cout << findPermutation(n, r) << endl;
18 }
```

Explanation: Nested functions mean a function relying on another function to fulfill its logic. `findPermutation` handles the nPr logic but defers the actual iteration math to `getFactorial`.

.....

All the best! Keep Coding ♡